

# On Optimizing Distributed Transactions in a Multi-region Data Store (PVLDB'25)

Haoze Song, Yongqi Wang, Xusheng Chen, Hao Feng, Yazhi Feng,  
Xieyun Fang, Heming Cui, Linghe Kong



The University of Hong Kong



Shanghai Jiao Tong University

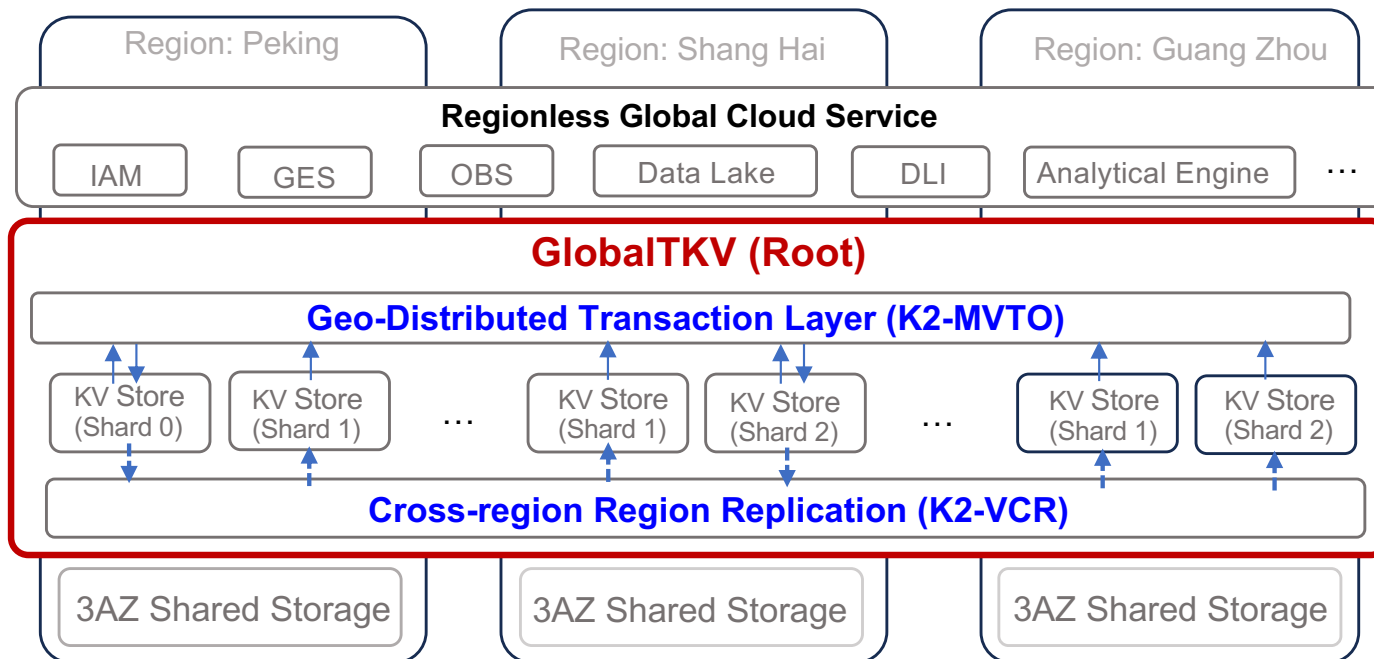


Huawei Technologies Co., Ltd.

# Background: GlobalTKV Project

- The design and implementation of our paper is based on GlobalTKV.  
(i.e., Huawei Cloud's next-generation data foundation for supporting root service)

## GlobalTKV's vision

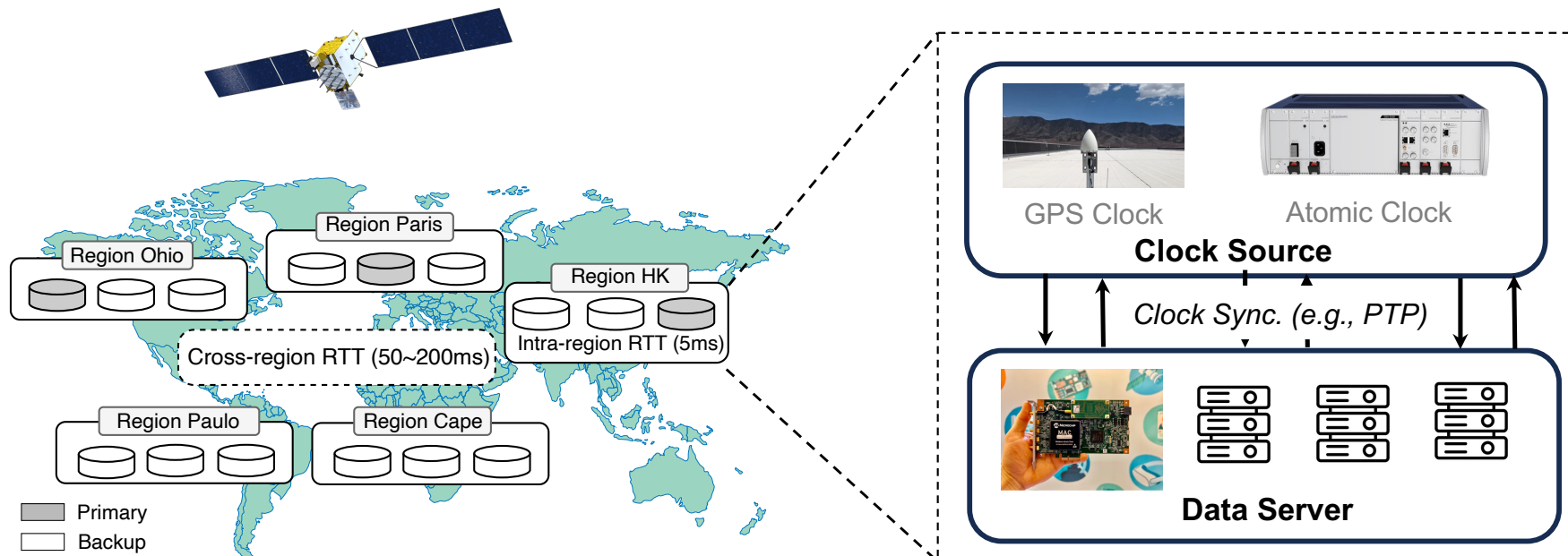


## Goals?

- ☐ Strict Consistency.
- ☐ Global Transaction (ACID).
- ☐ Near-data Processing.
- ☐ High Availability.
- ☐ Strong Performance.

# Background: TrueTime Clock

- TrueTime combines two sources to ensure reliable timestamps:
  - Atomic Clocks: Minimize clock drift to ensure global time synchronization.
  - GPS Clocks: Use satellite-based time for accurate geolocation and timestamping.



# Motivation: TrueTime's power was not fully utilized

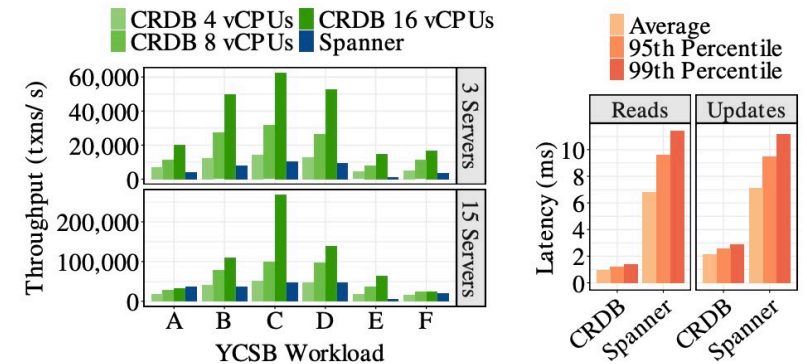
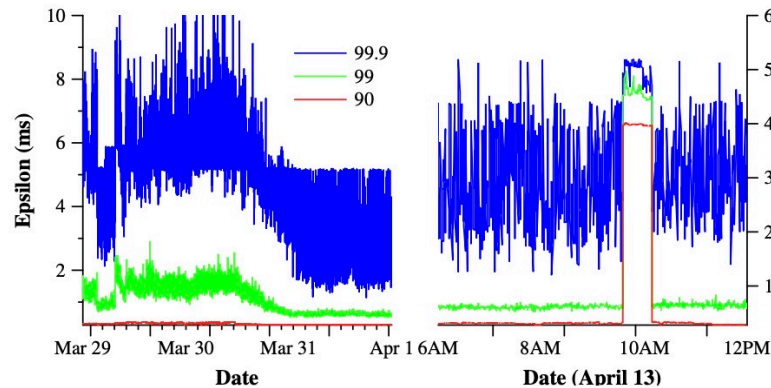


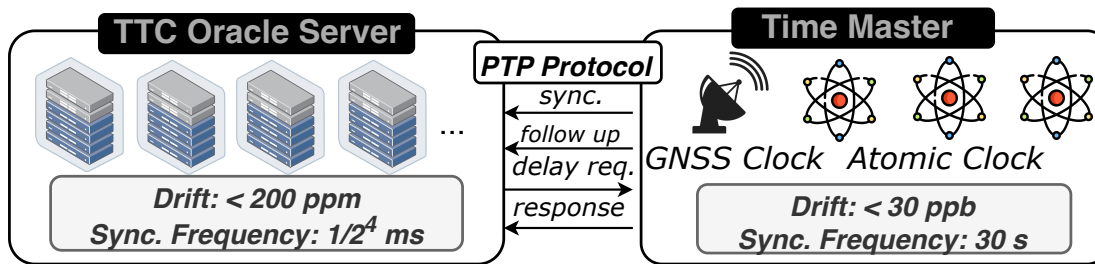
Figure 1: Time Uncertainty Bounds Reported in Spanner's paper [OSDI'12]      Figure 2: Unsatisfactory transaction performance [SIGMOD'20]

- **Figure 1:** In a real deployment, the time uncertainty bounds of **Spanner** can be larger than 10ms since the clock synchronization is heavy and can be blocked in network traffic.
- **Figure 2:** The performance of **Spanner** is sub-optimal even if compared to other baselines without TrueTime. Many research works report their performance magnitude better than Spanner (e.g., Tapir [SOSP'16], Detock [VLDB'24]).

**Observation:** (a). Large Uncertainty is caused by Large Scale TrueTime Clock Deployment (e.g., more than 100 nodes) (b). TrueTime is not fully utilized in Spanner's core transaction protocol.

# A new Primitive: Timestamp Batching Algorithm

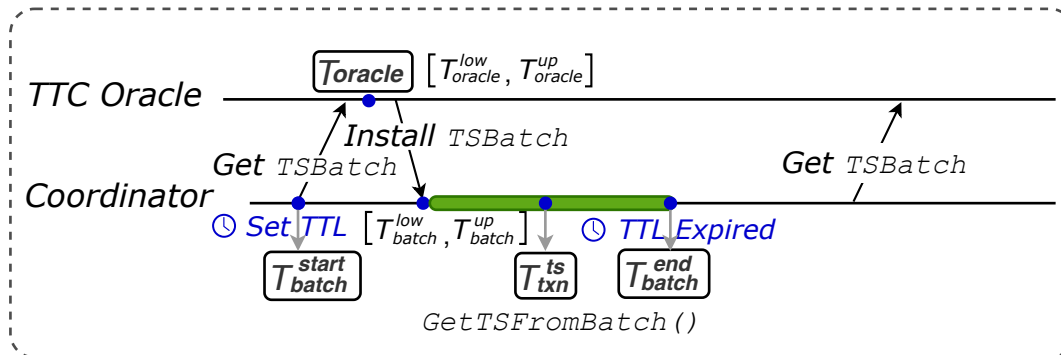
- Instead of equipping TrueTime clocks to all machines, we implement a TrueTime Oracle in each data center, which consists of a cluster of servers running precision timing protocol.



- ❑ A small cluster mitigates the bottleneck of limited synchronization channels on the time master.
- ❑ A stand-alone PTP cluster simplifies our fault-tolerance model and further reduces software and network jitters in clock synchronization

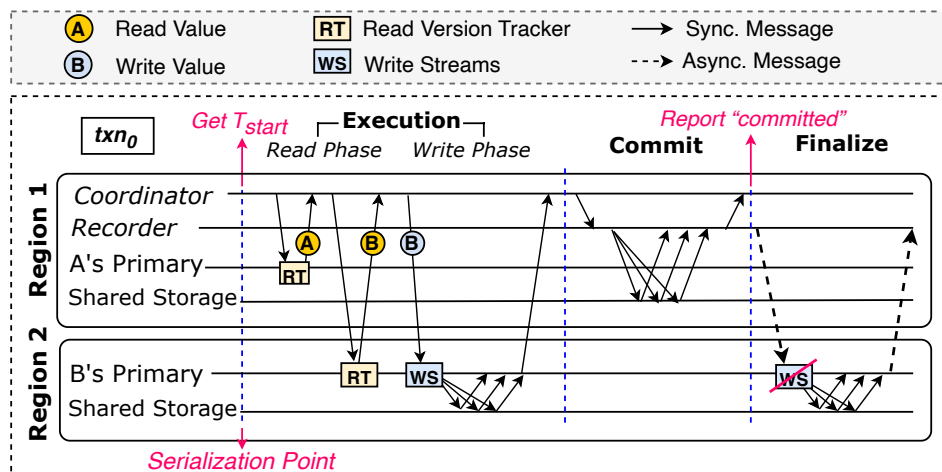
## Wait! How about correctness?

- Hybrid timestamp: Physical part + Logical Part.
- Using the true upper bounds for physical time.
- Commit a transaction by waiting out uncertainty.



# Realizing multi-version properties for read-write transactions

- We redesigned existing multi-version timestamp ordering protocol to realize the full potential of TrueTime.



## Key Features?

- ☐ Using TrueTime for transaction ordering.
- ☐ Request timestamp for each transaction only once.
- ☐ Using one intra-region RTT for committing
- ☐ Write-Write Conflicts free

## K2 Transaction Data Node Protocol

### At Primary Data Node:

```

Read (txn, key) :
1: if exist WS[key].ts <= txn.startTS:
2:   PushWS (key, startTS)           ► resolve undetermined write (Part of Write-Read Conflict)
3: else:
4:   update txn.startTS to Read Version Tracker (RT)
5:   return the latest value version whose ts <= txn.startTS
    
```

```

Write (txn, key, value) :
6: if exist RT[key].ts > txn.startTS:           ► resolve Read-Write conflict
7:   Abort (txn)
8: else:
9:   add txn.startTS, value to Write Streams (WS)
10:  return success, Epoch_i ← GetEpoch ()
    
```

```

PushWS (key, txn.startTS) :
11: for each w in WS[key]:
12:   ask w's recorder for decision
13:   if abort:
14:     clean and ignore w
15:   return the latest value version whose ts <= txn.startTS
    
```

```

Finalize (txn, key, txn.Epoch) :
16: persist (txn.startTS, WS[key], Epoch_C)
17: garbageCollect (txn.startTS, WS[key])
    
```

### At Recorder:

```

Commit (txn, <epoch_i>) :
18: Epoch_R ← GetEpoch ()
19: Epoch_C ← Max (Epoch_i, Epoch_R)           ► Epoch_i is advised by Data Node_j in write ()
20: Persist (txn, Epoch_C)
21: for each write operation:
22:   Node_i.Finalize (txn, key, Epoch_C)       ► async. finalize
    
```

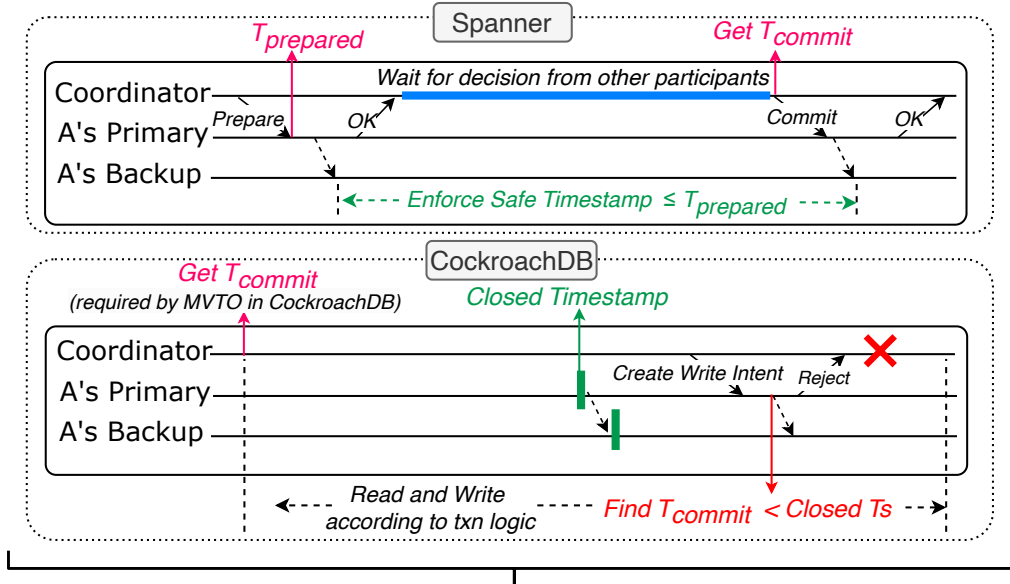
### At Replica Data Node:

```

RepRead (txn, key) :
23: hold until Replayed Epochs' End Timestamp >= txn.startTS
24: use the Ceiling Epoch of txn.startTS as a read view
25: if exist WS[key].ts <= txn.startTS:
26:   PushWS (key, startTS)
27: else:
28:   return the latest value version whose ts <= txn.startTS
    
```

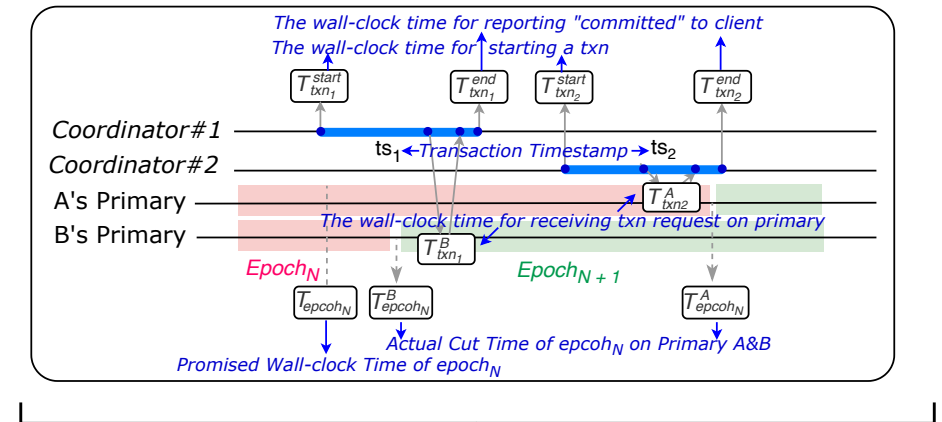
# TrueTime-based replica reads

- Existing approaches introduce mutual blocking between read-write transactions at primaries and visibility replay at replicas.



**Figure 1: Existing Approaches:**  
**Spanner [OSDI'12] and CockroachDB [SIGMOD'20]**

$t_{safe}^{TM}$  as defined above has a weakness, in that a single prepared transaction prevents  $t_{safe}$  from advancing. As a result, no reads can occur at later timestamps, even if the reads do not conflict with the transaction. Such



**Figure 2: Our Approach**

## Key Features?

- ❑ Using start timestamp for data versioning.
- ❑ Using TrueTime for generating wall-clock-time bounded Epoches. Each epoch is a snapshot step.
- ❑ Resolving undetermined writes by actively asking.



# Evaluation: Performance Overview

➤ We evaluated the performance of K2 with the two competitors (Spanner and CockroachDB).

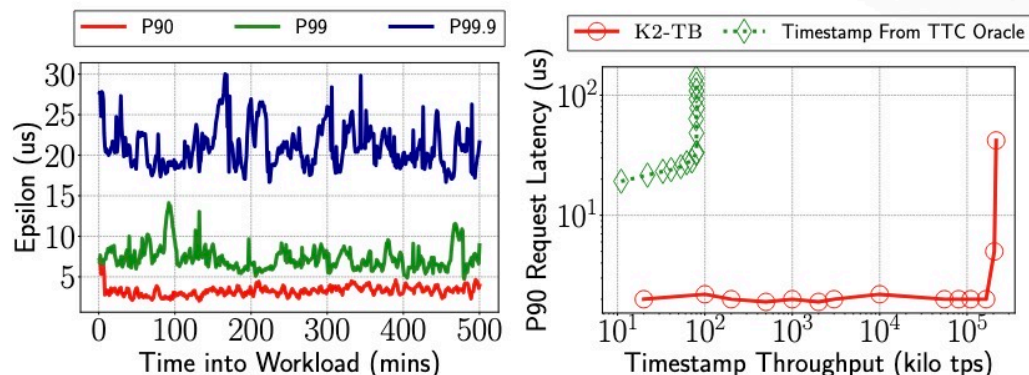


Figure 15: Epsilon over Time

Figure 16: Performance of K2-TB

Figure 1: Performance of TrueTime Oracle

## Highlight

- ❑ Sub-millisecond time uncertainty bounds (with batching).
- ❑ An order of magnitude better performance than Spanner and 2.32× better performance than CockroachDB in TPC-C.
- ❑ Sub-second visibility delay under a typical geo-distributed deployment without artificially aborting other transactions.

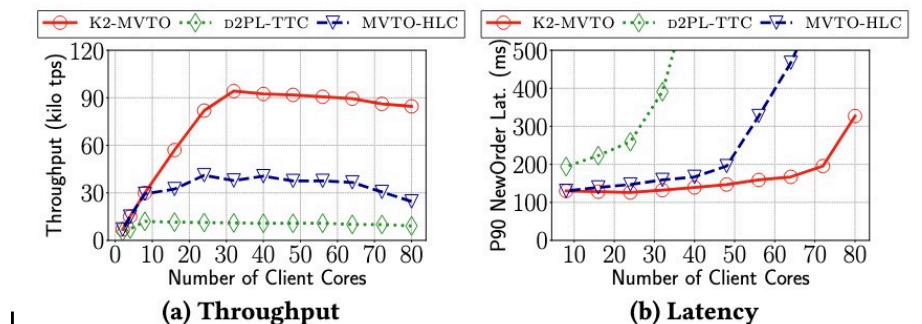


Figure 2: Transaction Performance

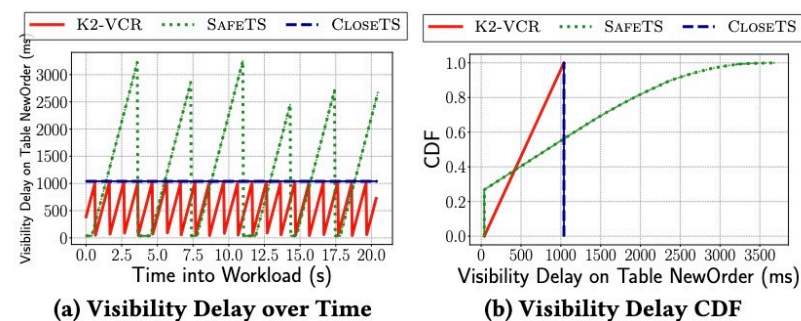


Figure 3: Replica Reads



# **Thanks for listening**

*Speaker: Haoze Song*